Révisions du Contrôle 1

Fondations d’Apache Kafka — Séances 1 à 7

ESGI • 5e année • Data Engineering • Séance de révision avant l’examen

SÉANCE 1

Le log, la fondation de Kafka

- Kafka est une **plateforme de streaming** bâtie sur un **log distribué**.
- Le **log** : messages *ajoutés en fin* (append-only), chacun reçoit un **offset** croissant ; immuable, persisté, **rejouable**.
- Différent d'une **file** : lire n'efface rien ; plusieurs consommateurs *indépendants* ; rejeu depuis n'importe quel offset.
- Découple **N producteurs** et **M consommateurs** (fini le maillage point à point).
- **KRaft** (depuis Kafka **4.0**) remplace ZooKeeper : métadonnées dans un quorum de contrôleurs.

✗ **Piège.** Le log n'est pas une file jetable : un message consommé *n'est pas supprimé*. D'où le rejeu et les lecteurs multiples.

SÉANCE 2

Topics, partitions, offsets, cluster

- Un **topic** (nom logique) est découpé en **partitions** = l'unité réelle de **stockage**, d'**ordre** et de **parallélisme**.
- Identité d'un message = **(topic, partition, offset)** ; l'offset est *propre à une partition*.
- L'**ordre** n'est garanti **qu'au sein d'une partition** — jamais globalement sur le topic.
- **Clé** → **partition** : `partition = murmur2(clé) % n`. Même clé → même partition → ordre par clé. Sans clé : sticky / round-robin.
- **RF = 3** : par partition, 1 *leader* + 2 *réplicas*. Le `bootstrap` est le point d'entrée vers le cluster.

✗ **Piège.** Augmenter le nombre de partitions d'un topic existant **casse l'ordre par clé** (une même clé peut changer de partition).

SÉANCE 3

Réplication & durabilité

- **ISR** = réplicas *synchronisés* (à jour), ensemble **dynamique**. L'élection du leader se fait *dans l'ISR*.
- `acks` : **0** (aucune attente, perte possible) • **1** (leader seul) • **all** (tout l'ISR).
- **Référence** : `RF=3` + `min.insync.replicas=2` + `acks=all` ⇒ tolère **1 panne**, **0 perte**.
- **Compaction** : garde la *dernière valeur par clé*. **Rétention** par temps/taille : suppression par *segment entier*.
- **Unclean election** = disponibilité au *prix d'une perte* ; **ELR** (4.x) réduit ces cas.

✗ **Piège.** `acks=all` *seul* ne suffit pas : sans `min.insync.replicas` adapté, une écriture confirmée peut encore être perdue.

SÉANCE 4

Producer & Consumer (API Java)

- Dépendance `kafka-clients`. `KafkaProducer`/`KafkaConsumer` sont `AutoCloseable` (try-with-resources).
- `send()` est **asynchrone** (Future + callback). `poll(Duration)` = modèle **pull**, en boucle.
- `auto.offset.reset=earliest` : *aucun* offset commité → lecture depuis le début ; sinon `latest`.
- Le `group.id` est la **mémoire du groupe** : nouveau `group.id` + `earliest` = **rejeu complet**.

Le rejeu, vu du code

```
// meme group.id            -> reprend au dernier offset commite
// nouveau group.id+earliest -> relit TOUT depuis le debut
var records = consumer.poll(Duration.ofMillis(100));
for (var r : records) traiter(r);
```

Consumer groups & commit

- Dans un groupe : **au plus 1 consommateur par partition** ; les consommateurs en trop sont *oisifs*.
- Un **rebalance** survient quand un membre arrive ou part (LAG transitoire, normal).
- `commitSync()` **après** traitement = **at-least-once** ; **avant** = **at-most-once**. On met `enable.auto.commit=false`.
- `group.protocol=consumer` (KIP-848) ; le *static membership* (`group.instance.id`) réduit les rebalances.

Commit manuel apres traitement = at-least-once

```
var records = consumer.poll(Duration.ofMillis(100));
for (var r : records) traiter(r);   // 1) on traite d'abord
consumer.commitSync();              // 2) puis on valide l'offset
```

SÉANCE 6

Les sémantiques de livraison

At-most-once	At-least-once	**Exactly-once**
Commit *avant*.	Commit *après*.	Idempotence
Perte possible.	**Doublons** possibles.	+ transactions.
		Ni perte ni doublon.

- **Idempotence** : par défaut depuis **Kafka 3.0** ; supprime les doublons de *retries* (`PID`+séquence).
- **Transactions** : écriture **atomique** multi-partitions ; `read_committed` vs `read_uncommitted`.

✗ **Piège.** L'exactly-once vaut **dans Kafka**. Vers une base externe → outbox / idempotence applicative.

SÉANCE 7

Schémas & Schema Registry

- Un message = des **octets** ; le **schéma** est le *contrat*. **JSON** (fragile) vs **Avro** (binaire compact, bonne évolution) vs Protobuf.
- Le **Schema Registry** est un composant **Confluent** (REST), *pas* Apache. `kafka-avro-serializer` vient de `packages.confluent.io`.
- **Trame** : `magic byte 0x00` + `ID (4 octets)` + payload. Le schéma *complet n'est pas* dans le message.
- Compat `BACKWARD` (défaut) : le nouveau lit l'ancien → ajouter un champ **avec défaut**, *consommateurs d'abord*. Sinon `409`.

Champ optionnel en Avro

Une **union** `["null","string"]` avec `default: null` : le champ peut être absent (compatible BACKWARD).

À GARDER EN TÊTE

Le mémo de durabilité

Producteur durable + idempotent

```
// cote producteur
props.put(ACKS_CONFIG, "all");
props.put(ENABLE_IDEMPOTENCE_CONFIG,
    true);

// cote topic / broker
//   replication.factor   = 3
//   min.insync.replicas  = 2
```

Ce que ça garantit

- ✓ Tolère la panne d'**1 broker**.
- ✓ **Aucune perte** confirmée.
- ✓ **Aucun doublon** (retries).

✗ Retirer `min.insync.replicas` ou baisser `acks` → la garantie tombe.

NE TOMBEZ PAS DEDANS

Les pièges classiques du contrôle

✗ **Ordre global.** L'ordre n'existe qu'*au sein d'une partition*.

✗ **`acks=all` seul.** Sans `min.insync.replicas`, zéro-perte non assuré.

✗ **Commit avant.** At-most-once : risque de perte.

✗ **Schema Registry.** Composant *Confluent*, pas Apache.

✗ **Trame Avro.** Le message porte un *ID*, pas le schéma.

✗ **Exactly-once externe.** Vaut dans Kafka ; sinon outbox.

Révisions — Contrôle 1

Bon courage !

Le jour de l’examen

Relisez les encadrés « piège », sachez *lire* un extrait de code Kafka, et gérez votre temps (45 min, 21 questions). Vous connaissez tout ça : faites-vous confiance.